

From Latin Digits to Babylonian Cuneiform: Measuring the Provenance of Number Helices in LLMs

Girish Gupta*

Abstract

Kantamneni & Tegmark (2025) (K&T) show that LLMs encode integers $a \in [0, 99]$ as a *generalised helix*: a linear “number-line” axis plus modular circles at $T \in \{2, 5, 10, 100\}$. A natural reading is that the transformer constructs this geometry through depth. We introduce a four-part diagnostic kit that asks where the geometry *comes from*: (i) the provenance ratio $\rho = R_{\text{helix}}^2(L=0) / \max_L R_{\text{helix}}^2(L)$, (ii) a basis-bandwidth and window-wrap check, (iii) a random-embedding control, and (iv) CKA between the $L=0$ and peak-layer helix subspaces. Applied across 8 models $\times$ 12 numeral scripts (96 cells), the kit shows that K&T’s central Latin result holds cleanly on their three models (Pythia-6.9B, Llama-3.1-8B, GPT-J-6B; $\rho = 0.34, 0.39, 0.41$) but that the apparent cross-script generalisation of the helix is largely *pre-transformer*: only 6–8 of 96 cells clear a joint (ρ , quality) bar for depth-built geometry. The clearest single result is on Babylonian cuneiform: replacing every learned embedding with a random vector at matched scale leaves the $L=0$ helix score unchanged on all 8 models ($|\Delta R^2| \leq 0.008$ in the native protocol). The measured base-60 geometry arises mechanically from how the renderer encodes wedges and how mean-pooling combines them, with no learned semantics required. Probes, SAEs, and any “the model represents X as a manifold” claim need a provenance check before attributing measured geometry to learning.

*girish@girishgupta.com

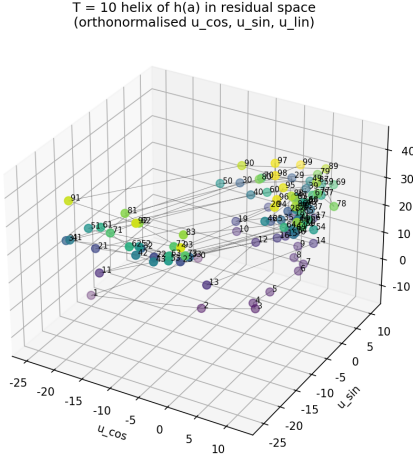


Figure 1: The number helix at $T = 10$ in Pythia-6.9B’s residual stream, Latin digits 0–99, peak layer. Two axes are the $(\cos 2\pi a/10, \sin 2\pi a/10)$ pair; the vertical axis is the linear “number-line” direction. Colour indicates a . Integers sharing the units digit land at the same angular position; magnitude lifts them along the vertical.

1. Introduction

Kantamneni & Tegmark (2025) reverse-engineer a striking representation: in the residual stream of three mid-sized LLMs (Pythia-6.9B, GPT-J-6B, Llama-3.1-8B), each integer $a \in [0, 99]$ is encoded as a generalised helix—a linear magnitude axis plus orthogonal modular circles at periods $T \in \{2, 5, 10, 100\}$ (Fig. 1 shows the $T = 10$ slice). The basis is itself the LLM-scale echo of Nanda et al. (2023), who reverse-engineered the trig solution in a tiny transformer trained on modular addition (itself an interpretation of the Power et al. (2022) grokking dynamics). K&T further show the helix is causally used for addition.

A high helix R^2 does *not* tell us where the geometry came from. It might be constructed by transformer depth, or inherited from the input pipeline (rendering, tokenization, embedding lookup, pooling), or induced by the measurement protocol itself (wrong basis, too-short input range). This question matters: in the literature, “the model represents X as a [helix/sphere/manifold]” is routinely read as a claim about what the transformer does, when it can in principle be a claim about how the inputs were prepared.

This provenance question is also motivated by behavioral work on number tokenization. Singh & Strouse (2024) show that arithmetic performance in frontier LLMs depends strongly on how numbers are tokenized, with systematic error patterns at token boundaries. Their result is behavioral; ours is representational. If tokenization can change arithmetic behavior, can the input pipeline also *create* the geometry we later interpret as a learned representation?

Contributions. We introduce a four-part diagnostic kit (§2) and apply it across 8 models $\times$ 12 numeral

scripts (§3). We report three findings (§4). (1) K&T’s Pythia/Llama/GPT-J Latin cells survive cleanly under the wider protocol; the depth-built finding holds where the original paper claimed it. (2) The “helix-in-script- X -too” generalisation is mostly pre-transformer: only 6–8 of 96 cells qualify as depth-built or depth-amplified under a joint ρ +quality criterion. (3) Babylonian cuneiform is the cleanest case of mechanically-produced geometry: swapping every learned embedding for a random vector leaves the $L=0$ helix score unchanged on all 8 models. We discuss implications for probing and SAE interpretability (§5).

Complementary causal work: Feucht, Haklay et al. (2026) identify a layer-18 base-10 addition module in Llama-3.1-8B that reads from exactly the Fourier features the helix basis fits, on exactly the cell our diagnostic marks as depth-built—answering the *what is the geometry used for?* question on a cell we mark as the geometry-was-actually-built one.

2. The Diagnostic Kit

We characterise a (model, script) cell along four axes. Let $H \in \mathbb{R}^{n \times d}$ be the residual stream at layer L over n integers, with $h(a)$ the row for integer a .

Helix basis and quality. The trig basis is

$$B(a) = [a, \cos(2\pi a/T), \sin(2\pi a/T)]_{T \in \mathcal{T}}. \quad (1)$$

We fit $H \approx BW$ by ordinary least squares and report the variance-weighted R^2 at every layer, plus the K-d PCA upper bound at the same dimensionality (Eckart–Young). The ratio $R^2_{\text{helix}}/R^2_{\text{PCA}}$ measures *quality*: how dominant the helix subspace is among all K-d fits.

Provenance ratio ρ . The core diagnostic:

$$\rho = \frac{R^2_{\text{helix}}(L=0)}{\max_L R^2_{\text{helix}}(L)}. \quad (2)$$

$L = 0$ is the residual stream *before any transformer block has run*—the output of token embedding, positional embedding, and mean-pooling over numeral sub-tokens. $\rho = 1$ means the helix is already present at $L=0$ and depth does not improve the score (inherited); $\rho \rightarrow 0$ means depth substantially builds it (depth-built). ρ alone is insufficient: a low ρ in a low-quality cell just means the basis fits little anywhere. We classify a cell as *depth-built* when $\rho \leq 0.55$ and $R^2_{\text{helix}}/R^2_{\text{PCA}} \geq 0.70$; *inherited* when $\rho \geq 0.80$ at quality ≥ 0.55 ; *weak* when quality < 0.55 ; *ambiguous* otherwise.

Basis bandwidth and window wrap. A trig basis can only fit periods it includes; a fitted period T requires the input window to wrap T several times. Babylonian (native $T = 60$) and hex (natural $T \in \{16, 32, 64, 256\}$) fail both bars at the paper defaults. Mitigation: widen

the window past several wraps and add the native period to $\mathcal{T}$.

Random-embedding control. Replace the model’s learned embedding matrix with $E \sim \mathcal{N}(0, 1/\sqrt{d})$ at matched scale, recompute the L=0 representation (rendering + tokenization + mean-pooling are unchanged), and re-fit the helix. A small $\Delta R_{L=0}^2 = R_{\text{learned}}^2 - R_{\text{random}}^2$ means the L=0 geometry is mechanical—the embedding table contributed nothing the renderer + pooling did not already produce.

Representation alignment (CKA). Linear CKA (Kornblith et al., 2019) between the helix-projected L=0 and peak-layer representations distinguishes two cases ρ conflates: *pass-through* (high ρ , high CKA) versus *rebuild* (high ρ , low CKA: depth produces a similar-quality helix with a substantially different integer-to-integer geometry). CKA is rotation- and rescaling-invariant.

3. Experimental Setup

Models (all base, non-instruction-tuned): Pythia-6.9B, GPT-J-6B, Llama-3.1-8B, Gemma-4-E4B, Gemma-4-31B, OLMo-3-32B, Qwen2.5-7B, Qwen2.5-32B. Three of these (Pythia, GPT-J, Llama) are the original K&T set.

Scripts (12 total, four families); Table 1 gives each script’s rendering of 23 and its mathematical family. All inputs use mean pooling over numeral sub-tokens (last-token pooling produces a period-10 artifact for multi-token scripts; see App. A). The default protocol is $n_{\text{max}} = 100$, $\mathcal{T} = \{2, 5, 10, 100\}$; Babylonian/binary/hex additionally run with wider windows and native bases (App. C).

For each (model, script) cell we run one forward pass per integer with `output_hidden_states`, mean-pool the numeral’s sub-token positions at every layer, fit the helix basis, and record per-layer R^2 and ρ . Code, configurations, and all 8 GB of intermediate JSON are public; see App. G.

4. Results

4.1. The paper-models / Latin cells survive cleanly

On K&T’s three models on Latin, ρ is well below the inherited threshold and quality is high: Pythia $\rho = 0.34$ ($R_{\text{helix}}^2/R_{\text{PCA}}^2 = 0.85$), Llama $\rho = 0.39$ (0.85), GPT-J $\rho = 0.41$ (0.83). Less than half of the helix is at L=0; depth substantially builds it. These three cells are unambiguously depth-built under our criterion.

4.2. Cross-script generalisation is mostly pre-transformer

Figure 2 shows ρ for all 96 cells at paper-default protocol. The cool (depth-built) row is narrow: outside the

three paper-model Latin cells, depth substantially contributes only on Gemma’s strong non-Latin cells (Devanagari, Persian, Roman; $\rho = 0.44\text{--}0.63$). Most non-Latin cells on Pythia and Llama sit at $\rho = 0.73\text{--}1.00$: the helix is already present at L=0 because the tokenizer splits e.g. ४३ into per-digit sub-tokens whose mean varies smoothly with digit values, which the trig basis fits. Qwen takes pre-transformer geometry to its limit even on Latin: Qwen2.5-7B/Latin has $\rho = 1.00$ (helix peaks at L=0 exactly). Greek alphabetic numerals are universally weak ($\rho \geq 0.94$ on every non-Gemma model). The mutually-exclusive classifier (§2) marks 6 of 96 cells as depth-built and 4 as depth-amplified at paper defaults; the full grid is in App. F.

4.3. Babylonian: geometry from rendering alone

Babylonian cuneiform is the cleanest mechanical case. The natural protocol fix—widen n_{max} to 600 and add $T = 60$ to $\mathcal{T}$ —recovers 8–16 percentage points of helix R^2 uniformly across all 8 models. But the random-embedding control shows this recovered geometry is not learned. Table 2 gives the learned-vs-random $\Delta R_{L=0}^2$ on the native protocol; the maximum absolute gap is 0.0073. The L=0 Babylonian helix is mechanical: cuneiform renders n as additive wedge counts ($n=23 \rightarrow \llcorner \llcorner \llcorner$), the tokenizer splits those wedges, and mean-pooling produces a vector $h(a) \approx (t \cdot e_{\text{ten}} + u \cdot e_{\text{one}})/(t+u)$ that varies smoothly with a for *any* embedding vectors. The Fourier basis fits the smooth variation. This recapitulates the representational analogue of Singh & Strouse (2024): tokenization is not a neutral preprocessing detail but an inductive bias in the numerical reasoning pipeline.

For *contrast*, on Latin the gap is large and positive on every model ($\Delta = +0.03$ to $+0.18$; full table in App. D), and on Pythia/Llama/OLMo/GPT-J the random R^2 is ~ 0.08 (chance), so the learned L=0 helix on Latin (0.18–0.26) reflects learned digit semantics.

4.4. Pass-through versus rebuild

ρ alone conflates two mechanistically distinct cases that CKA separates. *Pass-through*: Qwen2.5-32B/Latin ($\rho = 0.89$, CKA = 0.91) and OLMo/Latin (0.63, 0.91)—depth leaves the row-similarity geometry untouched. *Rebuild*: Gemma/Latin ($\rho \approx 0.85$, CKA ≈ 0.34)—depth produces a similar-quality helix with a substantially different integer-to-integer geometry. On Babylonian, Pythia/Llama/OLMo have $\rho \approx 0.86$ but CKA = 0.25–0.53, so the peak Babylonian helix is depth-built on top of a mechanical L=0 starting point. The full (ρ , CKA) scatter is in App. E.

4.5. Binary and hex (one-paragraph summary)

Two more mode-1 case studies, two different stories. Binary ($n = 1024$, $\mathcal{T} = \{2, 4, 8, 16, 32, 64\}$) is mechanical like Babylonian: the random-embedding gap is $|\Delta| \leq 0.03$ on every model. Hex ($n = 1024$, $\mathcal{T} = \{16, 32, 64, 256\}$)

Table 1: The twelve numeral scripts, grouped by family. “Native T ” is the period a basis would have to include to fit the script’s positional structure exactly. The paper-default basis catches Latin and the other positional-base-10 scripts; binary/hex/Babylonian motivate extended-basis runs (App. C).

script	23 renders as	24 renders as	native T	family
Latin	23	24	10	positional base-10
Arabic-Indic	٢٣	٢٤	10	positional base-10
Persian	۲۳	۲۴	10	positional base-10
Devanagari	२३	२४	10	positional base-10
Thai	๒๓	๒๔	10	positional base-10
CJK positional	二三	二四	10	positional base-10
Binary	10111	11000	2–64	positional base-2
Hexadecimal	17	18	16–256	positional base-16
Greek alph.	κγ	κδ	—	additive (Milesian)
Hebrew alph.	כג	כד	—	additive (gematria)
Roman	XXIII	XXIV	—	additive
Babylonian	<<	<<	60	positional base-60, additive cols

	lat	arab	pers	deva	thai	chk	bin*	hex*	grk	heb	rom	bab*
Pythia-6.9B	.34	1.00	.99	.99	.98	.95	.51	.64	1.00	.95	.78	.86
GPT-J-6B	.41	1.00	.99	1.00	1.00	1.00	.51	.66	1.00	.96	.76	.85
Llama-3.1-8B	.39	1.00	.73	1.00	1.00	.99	.44	.77	1.00	1.00	.73	.87
Gemma-4-E4B	.89	.80	.63	.49	1.00	.96	.59	.43	.56	.80	.55	.89
Gemma-4-31B	.79	.74	.61	.44	.81	.88	.50	.34	.55	.75	.48	.88
OLMo-3-32B	.63	.92	.98	.85	1.00	1.00	.54	.77	1.00	1.00	.65	.82
Qwen2.5-7B	1.00	.89	.87	.77	.92	.91	.78	.73	.97	.90	.70	.93
Qwen2.5-32B	.89	.93	.91	.76	.88	.89	.64	.69	.94	.85	.67	.92

Figure 2: ρ at paper defaults ($n_{\max} = 100$, $\mathcal{T} = \{2, 5, 10, 100\}$). Teal = depth-built ($\rho \leq 0.45$); light teal = depth-amplified (0.45–0.60); cream = ambiguous; orange = inherited (0.80–0.95); deep orange = $\rho > 0.95$. Starred columns are mismeasured under paper defaults; see Table 2 and App. C for native bases.

Table 2: Babylonian random-embedding control at the native protocol ($n_{\max} = 600$, $\mathcal{T} = \{2, 5, 10, 60, 100\}$). All eight $|\Delta| \leq 0.008$. The L=0 helix score replicates with random vectors of matched scale.

model	learned $R^2_{L=0}$	random $R^2_{L=0}$	Δ
Pythia-6.9B	0.569	0.568	+0.001
GPT-J-6B	0.566	0.567	−0.002
Llama-3.1-8B	0.517	0.513	+0.003
Gemma-4-E4B	0.623	0.620	+0.002
Gemma-4-31B	0.621	0.621	−0.001
OLMo-3-32B	0.545	0.552	−0.007
Qwen2.5-7B	0.588	0.590	−0.002
Qwen2.5-32B	0.587	0.592	−0.004

is depth-built on the paper models: native-basis $\rho = 0.29, 0.30, 0.35, 0.35$ on Pythia/Llama/GPT-J/OLMo, and the random-embedding gap is positive on every model. The natural hypothesis (hex exposure in pre-training code/addresses/hashes) is not directly tested by our data. Full native-basis tables in App. C.

5. Discussion

The contribution here is the diagnostic kit, not the matrix. Three problems can make the standard “fit a helix, report R^2 ” procedure answer the wrong question: two

hide real structure (wrong basis, too-short window), one credits the transformer with structure it did not build (ρ catches this), and CKA reveals a fourth—a high ρ does not always mean the geometry is untouched, sometimes depth rebuilds the integer-to-integer pattern with similar quality but different shape. The cells where K&T’s reading—“depth builds the helix”—is unambiguously correct are narrower than the generalisation suggests: 6 of 96 at paper defaults, concentrated on the three paper-model Latin cells and Gemma’s strong non-Latin scripts.

The Babylonian random-embedding result is the sharpest version of a general warning: *some apparent neural geometry can come from rendering statistics alone*, before learned embeddings or transformer blocks are needed. Smooth, low-dimensional, easy-to-fit manifolds can be artefacts of how you encode the input. For number helices, ρ +CKA + the two spectral checks are the cheapest version of the provenance question; for other geometric claims (probe-discovered features, SAE directions, “the model represents X as a manifold”), analogues are needed before attributing measured geometry to learning.

The diagnostic does not invalidate the literature: K&T’s Latin results survive cleanly, and the replication across eight models shows the helix is architecturally general.

The Babylonian base-60 result is then a substantive statement about input pipelines, even if not about transformers. Causal work like Feucht, Haklay et al. (2026) answers the follow-up *what does the model do with the geometry?* on exactly the cells our diagnostic marks as depth-built.

References

- Kantamneni, S. and Tegmark, M. Language models use trigonometry to do addition. *arXiv preprint arXiv:2502.00873*, 2025.
- Nanda, N., Chan, L., Lieberum, T., Smith, J., and Steinhart, J. Progress measures for grokking via mechanistic interpretability. In *ICLR*, 2023. arXiv:2301.05217.
- Power, A., Burda, Y., Edwards, H., Babuschkin, I., and Misra, V. Grokking: Generalization beyond overfitting on small algorithmic datasets. *arXiv preprint arXiv:2201.02177*, 2022.
- Singh, A. K. and Strouse, D. J. Tokenization counts: the impact of tokenization on arithmetic in frontier LLMs. *arXiv preprint arXiv:2402.14903*, 2024.
- Feucht, S., Haklay, T., et al. Arithmetic in the wild: causal base-10 features in Llama 3. *arXiv preprint arXiv:2605.01148*, 2026.
- Kornblith, S., Norouzi, M., Lee, H., and Hinton, G. Similarity of neural network representations revisited. In *ICML*, 2019.

A. Methods details

Activation collection. For each integer a , we feed “<bos> format_number(a , script)” through the model with `output_hidden_states=True` and pool the numeral’s sub-token positions at every layer. We use mean pooling throughout; last-token pooling produces a period-10 artefact for multi-token scripts because numbers 23, 33, 43, ... all share the same last sub-token in per-digit BPE.

Helix fit. For numbers $\{0, \dots, n_{\max} - 1\}$ and periods $\mathcal{T}$, we construct B of shape $(n_{\max}, 1 + 2|\mathcal{T}|)$ from Eq. 1, then solve $H \approx BW$ by ordinary least squares with intercept. Report variance-weighted R^2 . The Eckart–Young upper bound is the variance-weighted reconstruction R^2 of the best $(1 + 2|\mathcal{T}|)$ -d PCA of H ; the helix/PCA ratio is our quality metric.

ρ and the peak layer. We collect activations at every `hidden_states[L]` (one forward pass per a), so $L = 0$ is the embedding output (after the token + positional embeddings and pooling, before any transformer block). We compute the helix R^2 at every L and define ρ from Eq. 2.

Random-embedding control. For a model with embedding matrix $E \in \mathbb{R}^{V \times d}$ we draw $\tilde{E} \sim \mathcal{N}(0, 1/\sqrt{d})$ (matched per-coordinate scale), use the model’s actual tokenizer to obtain sub-token id sequences for each a , mean-pool $\tilde{E}$ rows over those sub-tokens to obtain $\tilde{h}(a)$, fit the helix on $\tilde{H}$, and compare $\tilde{R}^2$ to the learned $R^2_{L=0}$. The tokenizer + renderer + pooling are identical to the learned condition; only the embedding lookup is replaced. No transformer blocks are run.

Linear CKA. For mean-centred $X \in \mathbb{R}^{n \times d_x}$, $Y \in \mathbb{R}^{n \times d_y}$, $\text{CKA}(X, Y) = \|X^\top Y\|_F^2 / (\|X^\top X\|_F \cdot \|Y^\top Y\|_F)$. We report CKA on the helix-projected representations $X = BW^{(L=0)}$, $Y = BW^{(\text{peak})}$, isolating whether the *helix slice* of the residual stream is the same at the two depths, not whether the rest of the model state is.

Classification thresholds. Mutually exclusive criteria: *depth-built* requires $\rho \leq 0.55$ and helix/PCA ≥ 0.70 ; *depth-amplified* requires $0.55 < \rho \leq 0.70$ and helix/PCA ≥ 0.60 ; *inherited* requires $\rho \geq 0.80$ at helix/PCA ≥ 0.55 ; *weak* is helix/PCA < 0.55 ; *ambiguous* is everything else (e.g. $0.70 < \rho < 0.80$). The bar is deliberately strict; relaxing helix/PCA to 0.60 adds Gemma’s two Roman cells.

B. Per-model peak layers on Latin

Where the helix lives varies dramatically across models. Pythia ($n_L = 32$) rises monotonically and peaks at the literal final layer. GPT-J ($n_L = 28$) does the same. Llama-3.1-8B ($n_L = 32$) peaks mid-stack at L=15. Gemma-4-E4B ($n_L = 42$) shows two regimes with a broad peak at L=17. Gemma-4-31B ($n_L = 60$) holds ~ 0.5 across most of the stack with a sharp spike to 0.70 at L=29. OLMo-3-32B ($n_L = 64$) rises through mid-stack to a peak at L=23. Qwen2.5-7B/Latin peaks at the literal embedding output (L=0; the helix is complete *before any block has run*). Qwen2.5-32B peaks at L=3 of 64. “Read the middle layer” is a Pythia-ism; for any new model, a layer sweep is mandatory before defining ρ .

C. Binary and hexadecimal: native-basis case studies

Binary (Table 3). At $n = 1024$ with $\mathcal{T} = \{2, 4, 8, 16, 32, 64\}$, fixing the basis recovers +0.10 to +0.37 helix R^2 over the paper basis at the same window. The biggest gains are on Qwen and Gemma, whose tokenizers split numerals into smaller sub-tokens, making the per-bit count structure more visible after mean-pooling. The random-embedding gap is $|\Delta R^2| \leq 0.03$ on every model: binary’s L=0 helix is mechanical, like Babylonian’s. Pythia/Llama/GPT-J/OLMo’s moderately-low native-basis ρ of 0.62–0.77 reflects depth doing additional work on top of a mechanical L=0, not learned bit semantics; the corresponding CKA values are 0.55–0.71.

Hexadecimal (Table 4). At $n = 1024$ with $\mathcal{T} = \{16, 32, 64, 256\}$, ΔR^2 is +0.19 to +0.29. The native-basis ρ on Pythia/Llama/GPT-J/OLMo is 0.29–0.35—in the same regime as the original paper’s Pythia/Latin (0.34) and Llama/Latin (0.39). Hex is the *second-cleanest* depth-built case in the matrix on those four models. The random-embedding gap is positive on every model (small on Llama/OLMo, sizeable on Gemma), so unlike binary the L=0 hex helix is partly learned. Gemma is more protocol-dependent: its paper-default on hex is 0.34 and 0.43 (in the depth-amplified regime), but its native-basis ρ rises to 0.54 and 0.63.

Table 3: Binary, paper basis vs. binary-native, $n = 1024$.

model	paper R^2	native R^2	Δ	native ρ
Pythia-6.9B	0.11	0.24	+0.13	0.62
GPT-J-6B	0.11	0.25	+0.13	0.71
Llama-3.1-8B	0.19	0.29	+0.10	0.73
Gemma-4-E4B	0.30	0.63	+0.32	1.00
Gemma-4-31B	0.34	0.63	+0.29	1.00
OLMo-3-32B	0.16	0.27	+0.11	0.77
Qwen2.5-7B	0.27	0.64	+0.37	1.00
Qwen2.5-32B	0.28	0.65	+0.37	1.00

Table 4: Hexadecimal, paper basis vs. hex-native, $n = 1024$.

model	paper R^2	native R^2	Δ	native ρ
Pythia-6.9B	0.05	0.29	+0.25	0.29
GPT-J-6B	0.06	0.27	+0.21	0.35
Llama-3.1-8B	0.05	0.34	+0.29	0.30
Gemma-4-E4B	0.16	0.36	+0.20	0.63
Gemma-4-31B	0.19	0.38	+0.19	0.54
OLMo-3-32B	0.06	0.26	+0.20	0.35
Qwen2.5-7B	0.06	0.35	+0.29	0.50
Qwen2.5-32B	0.07	0.36	+0.29	0.46

D. Random-embedding control across all scripts

Table 5 reports $\Delta R_{L=0}^2 = R_{\text{learned}}^2 - R_{\text{random}}^2$ for the paper-default protocol (extended-basis Babylonian/binary/hex deltas are in App. C and §4.3). Latin is large and positive on every model; Babylonian is near zero on every model. Per-digit positional scripts (Arabic-Indic, Persian, Devanagari, Thai) sit between: small positive Δ on Pythia/Llama/GPT-J/OLMo, larger positive on Gemma/Qwen whose tokenizers split digits finer (so even random embeddings carry more structure once mean-pooled, raising the floor without eliminating the learned contribution).

Table 5: Learned-vs-random $\Delta R_{L=0}^2$ across all paper-default cells. Negative or near-zero Δ marks mechanically-produced L=0 geometry.

script	Py	GPT-J	Llama	Gem-E4B	Gem-31B	OLMo	Qw-7B	Qw-32B
latin	+0.10	+0.12	+0.15	+0.12	+0.12	+0.18	+0.11	+0.03
arabic	+0.01	+0.01	+0.06	+0.13	+0.11	+0.03	+0.05	+0.04
persian	+0.01	-0.01	+0.09	+0.13	+0.11	+0.03	+0.03	+0.01
devanagari	+0.01	+0.00	+0.06	+0.13	+0.09	+0.03	+0.03	+0.03
thai	+0.01	+0.01	+0.01	+0.05	+0.04	+0.03	+0.02	+0.03
chinese	+0.03	+0.02	+0.07	+0.16	+0.14	+0.04	+0.03	+0.03
binary	+0.01	+0.02	-0.02	+0.05	+0.05	-0.03	+0.03	-0.01
hexadecimal	+0.03	+0.04	+0.04	-0.01	-0.02	+0.05	-0.01	-0.01
greek	+0.02	+0.02	+0.02	+0.04	+0.03	+0.03	+0.02	+0.02
hebrew	+0.02	+0.03	+0.01	+0.03	+0.03	+0.03	+0.06	+0.06
roman	+0.04	+0.05	+0.04	+0.15	+0.11	+0.03	+0.05	+0.04
babylonian	+0.00	-0.01	+0.01	+0.01	+0.01	-0.01	-0.00	-0.01

E. (ρ, CKA) scatter and the four quadrants

We compute linear CKA between the helix-projected L=0 representation and the helix-projected peak-layer representation for every cell. Four quadrants emerge in the (ρ, CKA) plane.

Pass-through (high ρ , high CKA): Qwen2.5-32B/Latin ($\rho = 0.89$, $\text{CKA} = 0.91$), OLMo/Latin (0.63, 0.91), Pythia $\times$ Persian/Devanagari/Chinese ($\text{CKA} \geq 0.98$). **Depth refines** (low-to-moderate ρ , high CKA): Pythia/Latin (0.34, 0.79), Llama/Latin (0.39, 0.81), Roman on every non-Gemma model ($\text{CKA} = 0.77\text{--}0.90$). **Rebuild** (high ρ , low CKA): the entire Gemma family ($\text{CKA} = 0.21\text{--}0.65$ across most scripts), Babylonian on Pythia/Llama/OLMo ($\rho \approx 0.86$, $\text{CKA} = 0.25\text{--}0.53$), Qwen on Devanagari ($\text{CKA} \approx 0.24\text{--}0.30$). **Depth builds new** (low ρ , low CKA):

rare; Gemma’s Devanagari/Persian/Roman/Greek cells edge into it ($\rho \approx 0.44\text{--}0.63$, $\text{CKA} \approx 0.29\text{--}0.57$).

The pass-through / rebuild distinction is the key new fact CKA contributes. The simpler reading—“Gemma/Latin $\rho \approx 0.85$, so mostly tokenizer”—was wrong. CKA on the same cell is only 0.33; depth produces a similar-quality helix with a substantially different integer-to-integer geometry. Across nearly every script, Gemma rebuilds. ρ alone made it look like Gemma was doing nothing; it is doing a lot, just not in a way that changes the score.

F. Classification grid (full)

The mutually-exclusive criteria of §2 applied to all 96 paper-default cells produce: depth-built = 6 (Pythia/Latin, GPT-J/Latin, Llama/Latin, Gemma-31B/Devanagari, Gemma-E4B/Devanagari, Gemma-31B/Binary—the last is in the protocol-mismeasured set); depth-amplified = 4 (OLMo/Latin, Gemma-31B/Persian, Gemma-E4B/Persian, Gemma-E4B/Binary); inherited = 35; weak = 36 (basis fits little); ambiguous = 15. Excluding the three protocol-mismeasured columns (binary, hex, Babylonian), the depth-built count is 5 out of 64.

G. Reproducibility

All code is public at https://github.com/girish/addition_helix (final URL on publication). Each driver script declares its dependencies via PEP 723 inline metadata and is runnable directly via `uv run`; one top-level `run.sh` executes the per-cell sweep (`main.py`), the aggregator (`aggregate.py`), the random-embedding control (`embed_control.py`), and the representation-alignment pass (`subspace_align.py`). The companion blogpost site (`www/`) reads the aggregated JSON and renders all figures interactively.

Compute. The full 96-cell sweep plus the three auxiliary passes completes in 6–8 hours on a single Apple M5 Max with 128 GB unified memory (MPS); fallback CUDA/CPU paths are in `main.py:pick_device`. The 32B-class models (Gemma-4-31B, OLMo-3-32B, Qwen2.5-32B) run in bf16. Each combination is idempotent: re-running skips any cell whose `fig_layer_sweep.json` already exists.

Data. All inputs are integers; we render them via `format_number` in `helix_lib.py` (12 scripts) and feed them through each model’s own tokenizer. No external corpus is used.